🎬 LP-0003 — script vidéo (~7 min)

> ⚠️ **Tap les liens directement sur ton phone — ne pas copy-paste (les URLs sont longues et peuvent être tronquées au copy).**

- Durée totale visée : **~7 minutes**
- Langue : English
- 🎬 `ACTION` = ce que tu fais à l'écran
- 💬 `SAY` = ce que tu lis à voix haute

> **Toutes les commandes de ce script ont été exécutées et vérifiées.** Les sorties annoncées sont les vraies. Si quelque chose ne sort pas comme écrit, arrête et dis-le-moi plutôt que d'improviser.

> **Le point délicat : l'explorer ne montre pas tout.** Son index est irrégulier et rate certaines de nos transactions, même une `create_distribution` publique, alors qu'elles sont toutes vivantes en RPC. Ça n'a rien à voir avec le design. Séparément, une transaction privacy ne publie ni `program_id` ni `instruction_data`, donc elle serait non-attribuable même avec un indexeur parfait, et ça c'est le sujet de la soumission. On ne mélange pas les deux : on lit la chaîne en RPC, plus fort qu'un explorer et qui ne peut pas t'afficher une page vide en pleine caméra.

⚙️ Pré-vol (~3 min de prep)

Terminal

🎬 **ACTION** :

```
cd /Users/eden/data/ns.com/lp-0003
clear
```

Agrandis la police (⌘+ plusieurs fois — le texte doit être lisible en 1080p), ferme Slack/Discord/notifications, fenêtre terminal en plein écran.

Un seul onglet browser

🎬 **ACTION : ONGLET A (le repo)** — tap ce lien :

→ **Repo : edenbd1/lp-0003-private-airdrop-distributor**

> **Pas d'onglet block explorer.** Deux raisons distinctes. Un, l'index de l'explorer est irrégulier et rate certaines de nos transactions, même une publique, donc une page "not found" en caméra ne prouverait rien. Deux, une transaction privacy est non-attribuable par construction (ni `program_id` ni `instruction_data`). On lit la chaîne en RPC à la place, à la scène 2 et à la scène 3, qui est la source de vérité.

Basecamp (pour la scène 4)

🎬 **ACTION** : Basecamp **déjà lancé et à jour**, avec le `.lgx` committé installé (la tuile `lp-0003-airdrop` visible dans la sidebar), et un dossier de distribution démo prêt à pointer. Réinstalle depuis le package committé avant d'enregistrer :

```
DST=~/Library/Application\ Support/Logos/LogosBasecamp/plugins/lp-0003-airdrop
rm -rf "$DST" && lgx extract app/lp-0003-airdrop.lgx --variant darwin-arm64 --
        output /tmp/x
mkdir -p "$DST" && cp -R /tmp/x/darwin-arm64/. "$DST/"
printf darwin-arm64 > "$DST/variant"
tar xzOf app/lp-0003-airdrop.lgx manifest.json > "$DST/manifest.json"
# puis relance Basecamp et vérifie que la tuile apparaît et que le claim marche
```

Vérif de dernière seconde (30 s, avant d'enregistrer)

🎬 **ACTION** :

```
CLAIM_TX=5cb5148572e92cced712a1e5af756b9f3e9b886e284c12f21324c6c8e5c0f9a0 \
NULLIFIER=dac661f3cf147e7cc3e88bf340e183702542bdb9c1756bccf21b86cab66ee96d \
DISTRIBUTION_ID=b1000000000000000000000000000000000000000000000000000000000000001
        \
./scripts/verify-onchain-claim.sh
```

Tu dois voir **cinq blocs OK** et la ligne finale `VERIFIED`. Si oui → QuickTime → Screen Recording → Démarre. Sinon → stop, préviens-moi.

SCÈNE 1 — Intro (0:00 – 0:45)

🎬 **ACTION** : Terminal vide

💬 **SAY** :

> “Hi, I’m Eden. This is my submission for Logos Lambda Prize L-P zero-zero-zero-three — a Private Allowlist and Airdrop Distributor.”

💬 **SAY** :

> “A distributor commits an eligibility set on chain. An eligible recipient claims their allocation — and the chain records only that a claim happened, never which address made it. An observer who knows every candidate address still cannot tell who claimed.”

💬 **SAY** :

> “The membership proof is genuinely verified on chain, and everything is live on the public L-E-Z testnet. Let me show you, starting from a clean clone.”

SCÈNE 2 — Le demo (0:45 – 3:15)

🎬 **ACTION** : Tape lentement :

```
./scripts/demo.sh
```

🎬 **ACTION** : Entrée. Laisse défiler, **puis scrolle doucement vers le haut** pendant que tu parles.

💬 **SAY** (pendant que ça tourne) :

> “This runs from a clean clone. No funded account, no local sequencer for the local steps.”

🎬 **ACTION** : Scrolle jusqu’en haut, sur `== 0. environment`

💬 **SAY** :

> “First line — R-I-S-C zero dev mode equals zero. Real proofs, no mock receipts. That’s required by the brief, and it’s the first thing I show.”

🎬 **ACTION** : Scrolle sur `== 1.`, `== 2.`, `== 3.`

💬 **SAY** :

> “Ten adversarial tests on the claim logic — non-members, borrowed Merkle paths, invented roots, forged nullifiers, inflated allocations, redirected destinations. Ten more on the encrypted bundle. Then six against the built verifier binary, run through the sequencer’s own executor — same executor, same input order, same thirty-two megabyte session limit the chain applies. A rejection you see there is the rejection the chain performs.”

🎬 **ACTION** : Scrolle sur `== 4.` et `== 5.`

💬 **SAY** :

> “Now the recipient flow. The distributor publishes one encrypted bundle, padded here to eight rows so it hides how many recipients there really are. A recipient opens it with only their own secret — they scan every row, skip the padding, and keep the one row whose data reconstructs a leaf that anchors to the committed root.”

🎬 **ACTION** : Scrolle sur `== 7. compute cost`

💬 **SAY** :

> “Measured compute cost: a claim is three hundred thirty-three thousand user cycles, one and a half percent of the public budget.”

🎬 **ACTION** : Scrolle sur `== 8. what an observer sees` — **ralentis ici**

💬 **SAY** (le cœur de la soumission — prends ton temps) :

> “This is the whole point. On chain, a claim is a single marker address. That address is a hash of a nullifier, and the nullifier is a hash of the recipient’s secret. Someone who knows every eligible address cannot compute that nullifier, so they cannot map this marker back to any of them. That is the unlinkability property.”

🎬 **ACTION** : Scrolle tout en bas, sur `== 9. the live deployment`

💬 **SAY** :

> “And the last step reaches the public testnet and verifies a claim that is actually deployed — five checks. The transaction is privacy-preserving, its receipt is a real Succinct STARK, not a dev-mode fake, and the marker is owned by the verifier program. That receipt could not exist unless a membership proof was verified on chain.”

SCÈNE 3 — Une preuve, générée en direct (3:15 – 5:15)

🎬 **ACTION** : Explique que le demo n'a pas *prouvé* — il a exécuté. Maintenant on génère une vraie preuve.

💬 **SAY** :

> "The demo executed the logic; it did not prove. Proving is the expensive part, so let me generate a real one now, live, with dev mode still off."

🎬 **ACTION** : Lance la génération d'une vraie preuve (compte financé requis ; `SIGNER`, `CLAIMANT` et `wallet` / `spel` déjà en place) :

```
export SIGNER=<ton public id financé> CLAIMANT=<ton private id autorisé>
./scripts/prove-one-claim.sh
```

💬 **SAY** (pendant que ça démarre) :

> "This commits one fresh distribution, then proves and submits a single real claim against it. Watch the proving step — about two and a half minutes on this laptop, because it generates a real STARK, and the privacy circuit then recursively verifies the chained call inside it. Dev mode is off, so these are real proofs."

> 🎬 **NOTE POST-PROD** : l'attente de proving (~2-3 min, ligne `proving locally...`) doit être **accélérée en post** (×8 à ×16), MAIS le terminal reste visible et continu — on ne coupe pas, on accélère, pour qu'il soit clair que rien n'est truqué.

🎬 **ACTION** : Le script imprime `proved + submitted in NNNs`, attend l'atterrissage, puis lance lui-même la vérification cinq-sur-cinq. Laisse-la s'afficher jusqu'à `VERIFIED`.

💬 **SAY** (quand le `VERIFIED` apparaît) :

> "There it is. A real proof, dev mode off, submitted on the privacy path, and the script reads it straight back off the chain: a Succinct STARK the sequencer verified, and the marker owned by the verifier. Two quick notes on the explorer. It does not show this transaction, but that is the explorer's index, not the chain: it also misses one of our public transactions, and the R-P-C returns all of them. And separately, a privacy transaction publishes no program id and no instruction data, so it is unattributable by design. That is why verification reads the chain directly."

SCÈNE 4 — L’app Basecamp (5:15 – 6:15)

🎬 **ACTION** : Passe sur Basecamp (déjà ouvert, la tuile LP-0003 dans la sidebar). Clique la tuile **lp-0003-airdrop**.

💬 **SAY** :

> “The same primitive, in a G-U-I. This is the L-P zero-zero-zero-three app, loaded in Logos Basecamp two point two — the real app, from the package committed in the repo, not a mockup.”

🎬 **ACTION** : La surface **Private Airdrop Claim** s’affiche. Pointe le champ distribution sur un dossier de distribution démo, choisis un destinataire, clique **Build claim**.

💬 **SAY** :

> “I point it at a distribution, pick a recipient, and build the claim. It prints a nullifier, a marker seed, and writes the claim args. And notice the C-L-I path field is empty: the app resolves the airdrop binary shipped inside the package itself, with dladdr, so a freshly installed package just works.”

💬 **SAY** :

> “The G-U-I and the chain compute the same commitments from the same code, because the app shells out to that same airdrop C-L-I. There is no second implementation to drift.”

SCÈNE 5 — L’audit (6:15 – 7:15)

🎬 **ACTION** : Reviens sur le terminal

💬 **SAY** :

> “One more thing, because I think it matters more than a feature list.”

💬 **SAY** :

> “After the first version was deployed, I ran an adversarial audit — reviewers and tests whose job is to *break* the code, not confirm it. It found a robustness gap in my own bundle-opening code.”

💬 **SAY** :

> “Opening a row used the raw Diffie-Hellman shared secret. A crafted low-order key would collapse that secret to the same value for every recipient, so one row injected into the open bundle would open for everyone and hand honest recipients garbage. It could never steal funds — the on-chain check rejects any forged claim — but it could grief a distribution.”

💬 **SAY** :

> “The fix rejects those non-contributory keys, and the recipient now keeps the first row that actually anchors to the committed root, not just the first that decrypts — so junk rows are skipped. Both halves are pinned by new tests, including one that runs through the deployed binary. I wrote the finding into the git history rather than quietly patching it.”

SCÈNE 6 — Closing (7:15 – 8:00)

🎬 **ACTION** : Passe sur l'ONGLET A (le repo)

💬 **SAY** :

> "To summarize. Two programs deployed on the public L-E-Z testnet, byte-identical to what's in the repository — you can check that from the deployment transaction hash. Three distributions committed, and twenty-three privacy-preserving claims landed and independently verifiable. Documented compute cost, a SPEL I-D-L, the Basecamp app you just saw loading and running, a demo script that runs from a clean clone, and green C-I with the adversarial tests running against the deployed binary on every push."

💬 **SAY** :

> "Repository at github dot com slash eden-b-d-one slash l-p dash zero zero zero three dash private dash airdrop dash distributor. The privacy model, with the residual leakage named rather than hidden, is in docs slash privacy dash model dot M-D. Thank you for reviewing."

🎬 **ACTION** : Attends 2 secondes en silence

🎬 **ACTION** : Stop l'enregistrement

📝 Post-recording

1. QuickTime → Export → 1080p mp4 → `lp-0003-submission.mp4`
2. Accélère la portion de proving de la scène 3 (×8 à ×16), terminal continu visible
3. YouTube Studio → tap **studio.youtube.com** → Upload → **Unlisted**
4. **Title** : `LP-0003 Private Allowlist / Airdrop Distributor — Lambda Prize submission (edenbd1)`
5. **Description** :

```
Submission demo for Logos Lambda Prize LP-0003 — Private Allowlist /
Airdrop Distributor.

A private airdrop on the Logos Execution Zone where a recipient claims an
allocation without revealing which address claimed, and the membership
proof
is genuinely verified on chain via LEZ's privacy-preserving transaction
path.

Repo:
https://github.com/edenbd1/lp-0003-private-airdrop-distributor

In that repo:
  docs/DEPLOYMENT.md      — every transaction hash, and how to re-verify
each
  docs/privacy-model.md   — the threat model, and what is deliberately not
hidden
  artifacts/e2e/claims.tsv — the 23 live claims

Public testnet:
https://testnet.lez.logos.co
```

6. Reviens avec **l'URL YouTube + "OK submit la PR"** → je prépare la PR, tu la relis, puis je l'ouvre

🆘 Cheat sheet — prononciation

- **LEZ** = “L-E-Z” (épelle)
- **SPEL** = “spell”
- **PDA** = “P-D-A” (épelle)
- **RISC0** = “risk zero”
- **nsk** = “N-S-K” (épelle)
- **npk** = “N-P-K” (épelle)
- **env::verify** = “env verify”
- **STARK** = “stark”
- **SHA256** = “SHA two fifty-six”
- **Diffie-Hellman** = “DIFF-ee HELL-man”
- **nullifier** = “NULL-ifier”
- **IDL** = “I-D-L” (épelle)
- **dladdr** = “D-L-addr” (épelle D-L, puis “adder”)
- **Basecamp** = “Basecamp”
- **.lgx** = “dot L-G-X” (épelle)
- **edenbd1** = “eden-B-D-one”

⏱️ Timing récap

Scène	De	À	Durée
1. Intro	0:00	0:45	45s
2. demo.sh	0:45	3:15	2m30
3. Preuve en direct	3:15	5:15	2m00 (proving accéléré)
4. L'app Basecamp	5:15	6:15	1m00
5. L'audit	6:15	7:15	1m00
6. Closing	7:15	8:00	45s
Total			**~8 min**

Entre 5 et 8 minutes c'est bon. Le brief demande une narration qui explique l'architecture et les décisions — pas un screencast muet. La scène 3 montre une vraie génération de preuve avec `RISC0_DEV_MODE=0`, et la scène 4 est ce qui te distingue : elle montre que tu audites ton propre travail.

Tu peux y aller. 🎬